

Calcolo Scientifico



[Python – parte 1]

Introduzione

Tipi di dati ed espressioni

Istruzione If e cicli While e For

Input e output



Python {

[Introduzione]

Python è un linguaggio di programmazione di alto livello, interpretato, orientato agli oggetti, ideato da Guido van Rossum all'inizio degli anni '90





Guido van
Rossum



|
}

Perchè usare Python {

[E' gratis!]



Python è completamente gratuito e nonostante ciò, da oltre 25 anni riceve costantemente miglioramenti che lo mantengono aggiornato e al passo coi tempi.

}

Perchè usare Python

[E' multi-paradigma!]



Python è un linguaggio multi-paradigma, che supporta sia la **programmazione procedurale** (che fa uso delle funzioni), sia la **programmazione ad oggetti**

1
2
3
4
5
6
7
8
9
10
11
12
13
14

}

}

Perchè usare Python {

[E' portabile!]



È possibile usarlo su diverse piattaforme
come: Unix, Linux, Windows, DOS, Macintosh,
Sistemi Real Time, OS/2, cellulari Android
e iOS.

}

Perchè usare Python {

[E' facile da usare!]



La sintassi e i diversi moduli e funzioni che sono già inclusi nel linguaggio sono consistenti, intuitivi, e facili da imparare, e il design del linguaggio si basa sul principio della "minor sorpresa": il comportamento del programma coincide con quanto ci si aspetta).

1
2
3
4
5
6
7
8
9
10
11
12
13
14

}

Perchè usare Python

[E' ricco di librerie!]



Include la standard library, cioè una collezione di oltre 200 moduli per svolgere i compiti più disparati. Inoltre, il Python Package Index consente di scaricare ed installare migliaia di moduli aggiuntivi creati e mantenuti dalla comunità.

1
2
3
4
5
6
7
8
9
10
11
12
13
14

}

}

Perchè usare Python {

[E' veloce!]



I programmi vengono automaticamente compilati in un formato chiamato **bytecode** prima di essere eseguiti. Questo formato è più compatto ed efficiente, e garantisce quindi prestazioni elevate.

}

Perchè usare Python

[E' integrabile con altri linguaggi!]



Oltre all'interprete classico scritto in C (**CPython**), esistono anche altri interpreti che consentono l'integrazione con diversi altri linguaggi (Jython integra Python e Java).

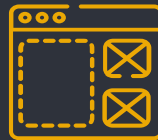
1
2
3
4
5
6
7
8
9
10
11
12
13
14

}

}

Cosa si può fare con Python {

- * GUI (Graphic User Interface)
- * Web Development
- * Game Development
- * Calcolo scientifico
- * Scripting
- * Cybersecurity
- * Data Science
- * Machine Learning e Deep Learning
- * Quantum Computing
- * e molto altro



Chi usa Python? {



E molti altri. . . .

}

Anaconda{

Anaconda è una distribuzione open source dei linguaggi di programmazione Python e R. Può essere installata su Windows, Linux e MacOS.

E' usata prevalentemente per il calcolo scientifico, Data Science, elaborazione dei Big Data e il Machine Learning.

Anaconda ha oltre 250 pacchetti automaticamente installati, e oltre 7,500 pacchetti open-source che possono essere installati dal [PyPI](#) (Python Package Installer). Include anche una GUI, **Anaconda Navigator** come alternativa grafica alla interfaccia da linea di comando(CLI).

}

Anaconda Navigator

 Upgrade Now

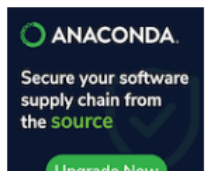
Connect 

 Home

 Environments

 Learning

 Community



End-to-end package security, guaranteed

Documentation

Anaconda Blog



Applications on

base (root)

Channels



CMD.exe Prompt

0.1.1

Run a cmd.exe terminal with your current environment from Navigator activated

Launch



DataLore

Online Data Analysis Tool with smart coding assistance by JetBrains. Edit and run your Python notebooks in the cloud and share them with your team.

Launch



IBM Watson Studio Cloud

IBM Watson Studio Cloud provides you the tools to analyze and visualize data, to cleanse and shape data, to create and train machine learning models. Prepare data and build models, using open source data science tools or visual modeling.

Launch



JupyterLab

 3.2.1

An extensible environment for interactive and reproducible computing, based on the Jupyter Notebook and Architecture.

Launch



Jupyter Notebook

 6.4.5

Web-based, interactive computing notebook environment. Edit and run human-readable docs while describing the



Powershell Prompt

0.0.1

Run a Powershell terminal with your current environment from Navigator activated



Qt Console

 5.1.1

PyQt GUI that supports inline figures, proper multiline editing with syntax highlighting, graphical calltips, and more.



Spyder

5.1.5

Scientific PYTHON Development Environment. Powerful Python IDE with advanced editing, interactive testing,



Scaricare e installare Anaconda {

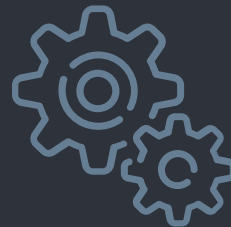
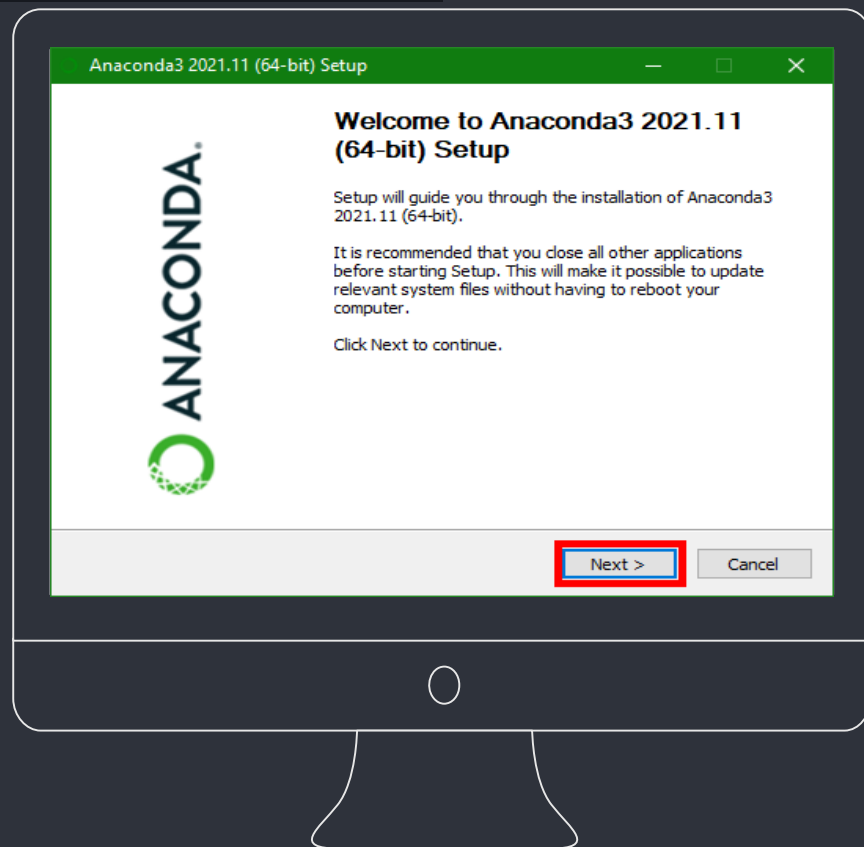
Link per scaricare Anaconda:

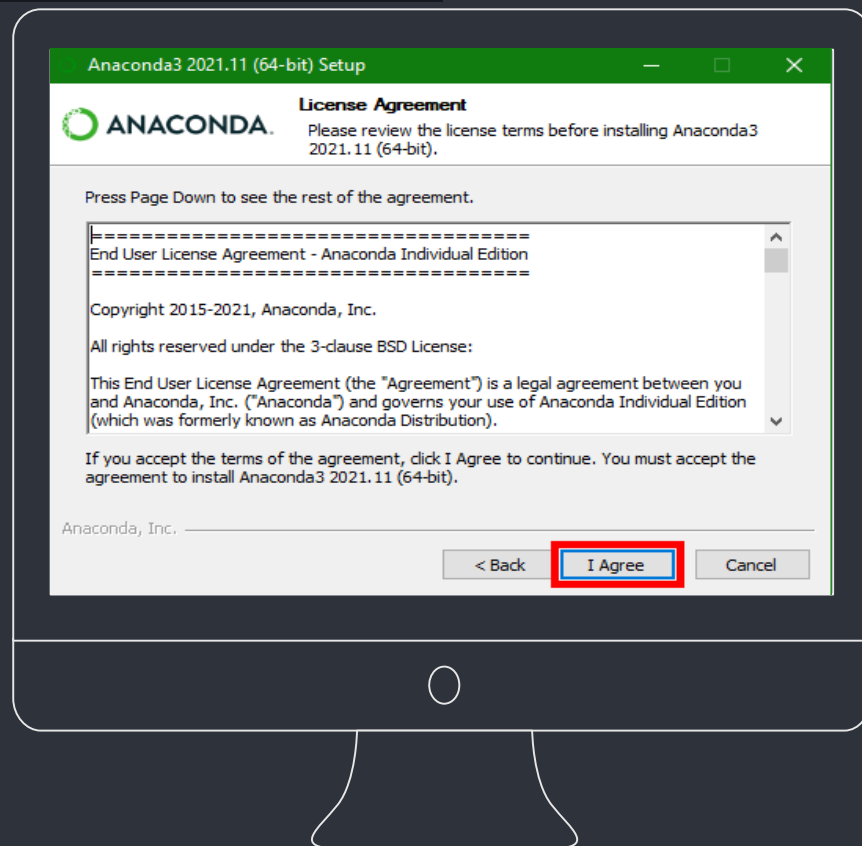
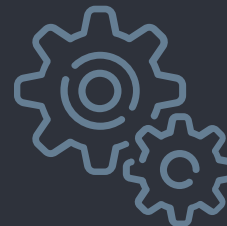
<https://www.anaconda.com/products/distribution>

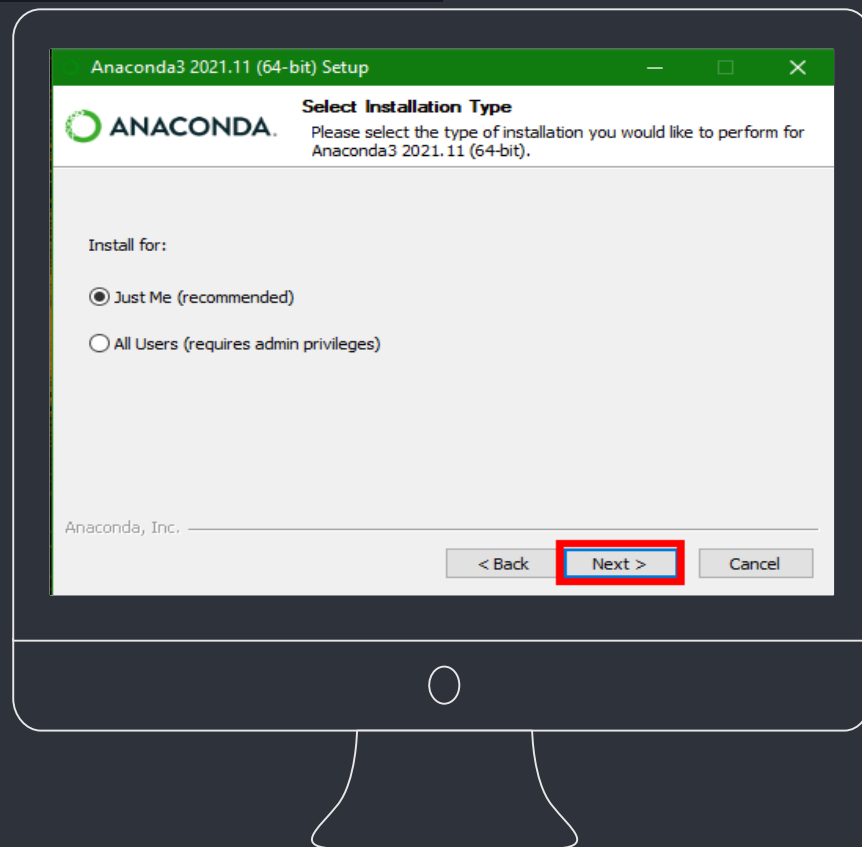
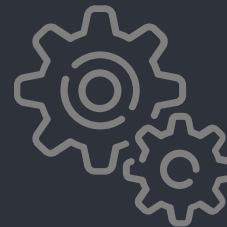
Una volta scaricato il file, lanciare il .exe e seguire le istruzioni del wizard.

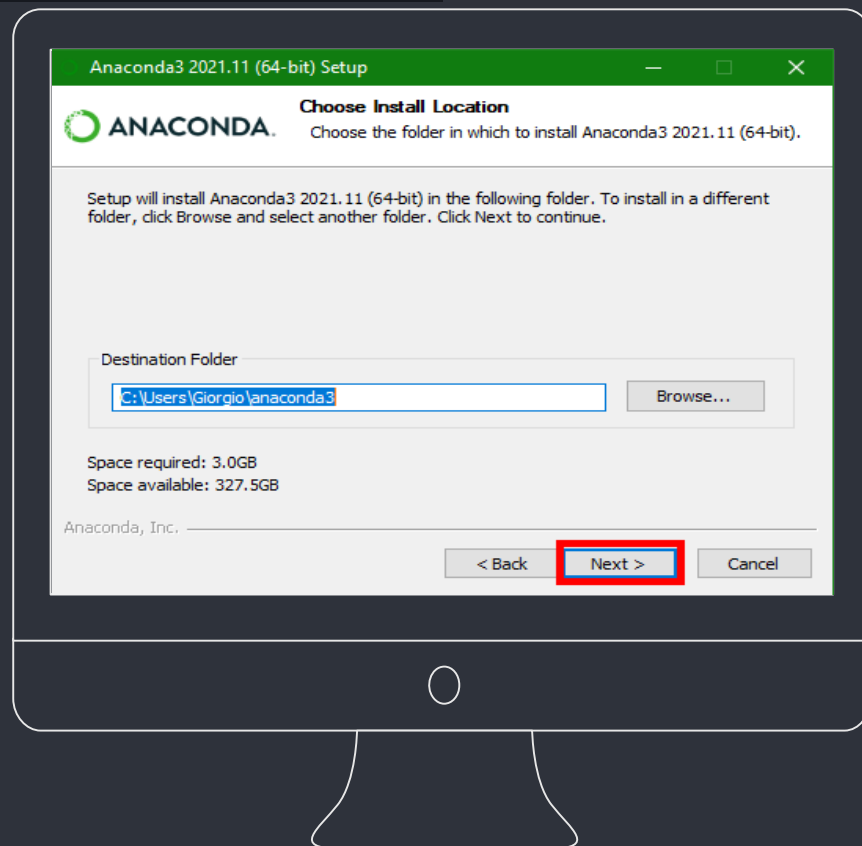
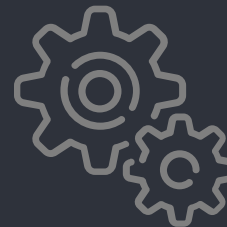
}

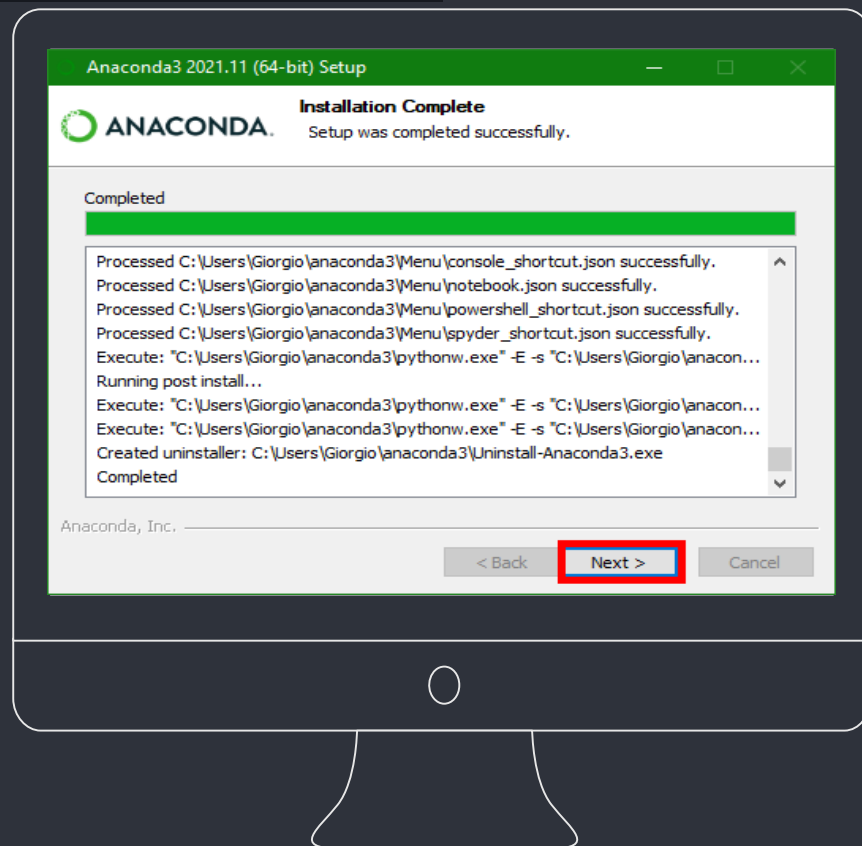
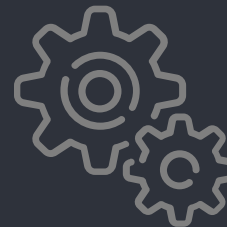
1
2
3
4
5
6
7
8
9
10
11
12
13
14

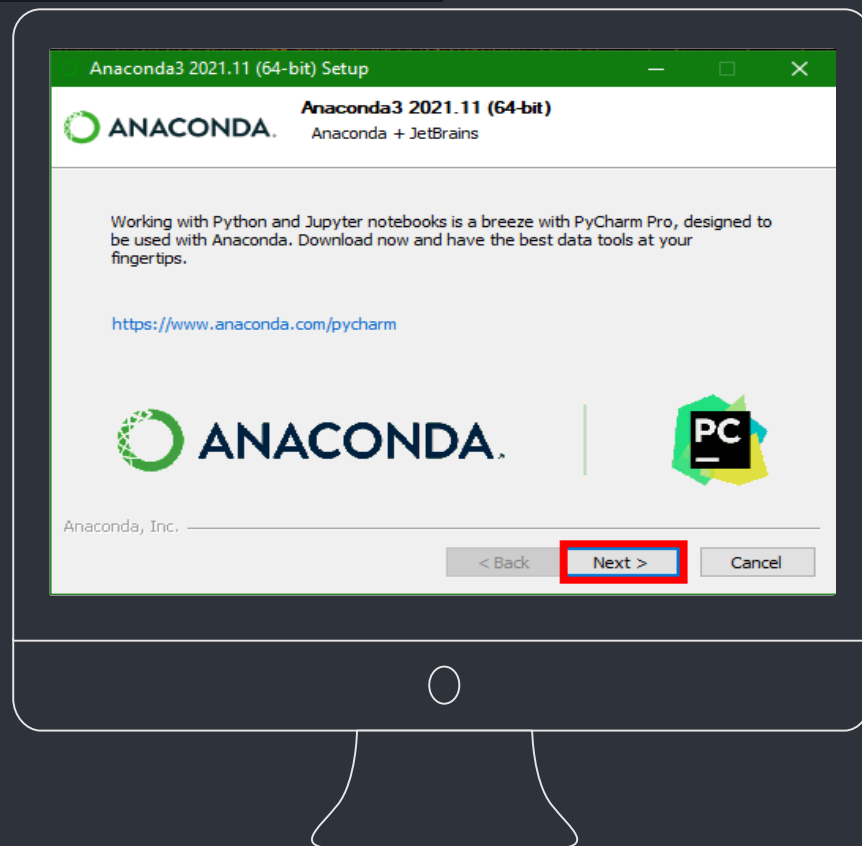
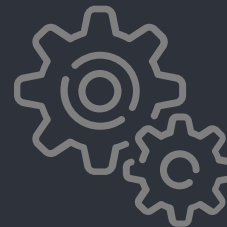






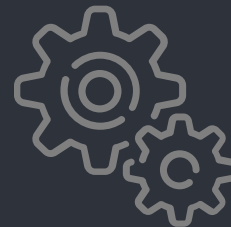


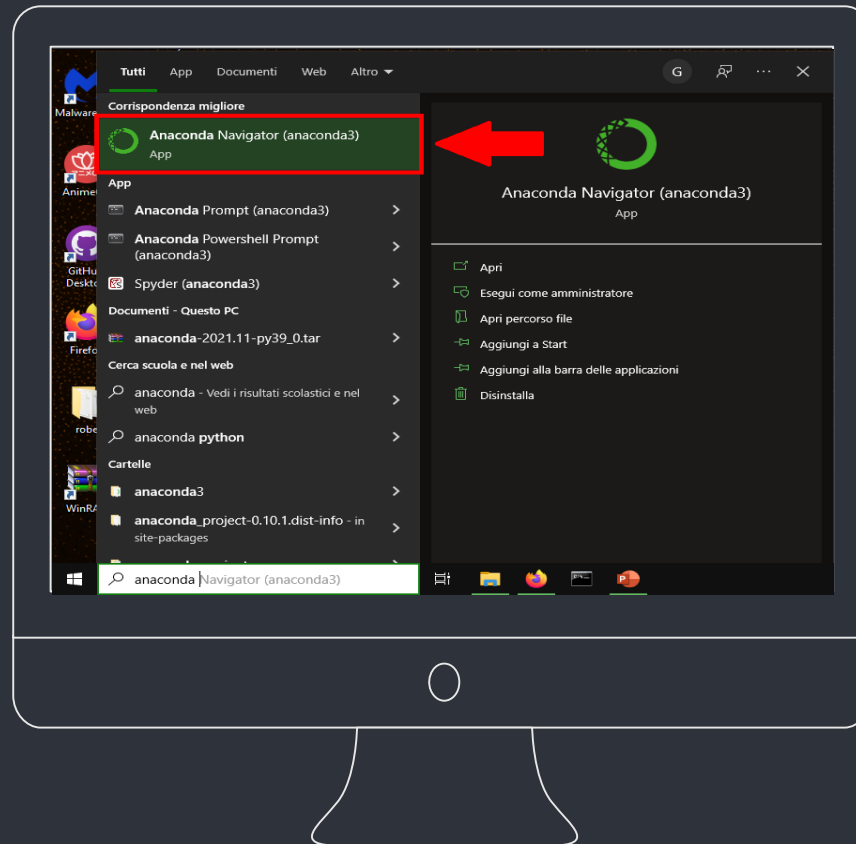
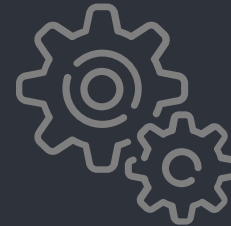




1
2
3
4
5
6
7
8
9
10
11
12
13
14

1
2
3
4
5
6
7
8
9
10
11
12
13
14





Spyder



Spyder (**S**cientific **PY**thon **D**evelopment **E**nvironment) è un ambiente di sviluppo integrato disponibile in Anaconda.



Home

Environments

Learning

Community

ANACONDA.

Secure your software supply chain from the source

Upgrade Now

End-to-end package security, guaranteed

Documentation

Anaconda Blog



Applications on

base (root)

Channels

environment from Navigator activated

Launch

coding assistance by JetBrains. Edit and run your Python notebooks in the cloud and share them with your team.

Launch

tools to analyze and visualize data, to cleanse and shape data, to create and train machine learning models. Prepare data and build models, using open source data science tools or visual modeling.

Launch

and reproducible computing, based on the Jupyter Notebook and Architecture.

Launch



Notebook

6.4.5

Web-based, interactive computing notebook environment. Edit and run human-readable docs while describing the data analysis.

Launch



Powershell Prompt

0.0.1

Run a Powershell terminal with your current environment from Navigator activated

Launch



Qt Console

5.1.1

PyQt GUI that supports inline figures, proper multiline editing with syntax highlighting, graphical calltips, and more.

Launch



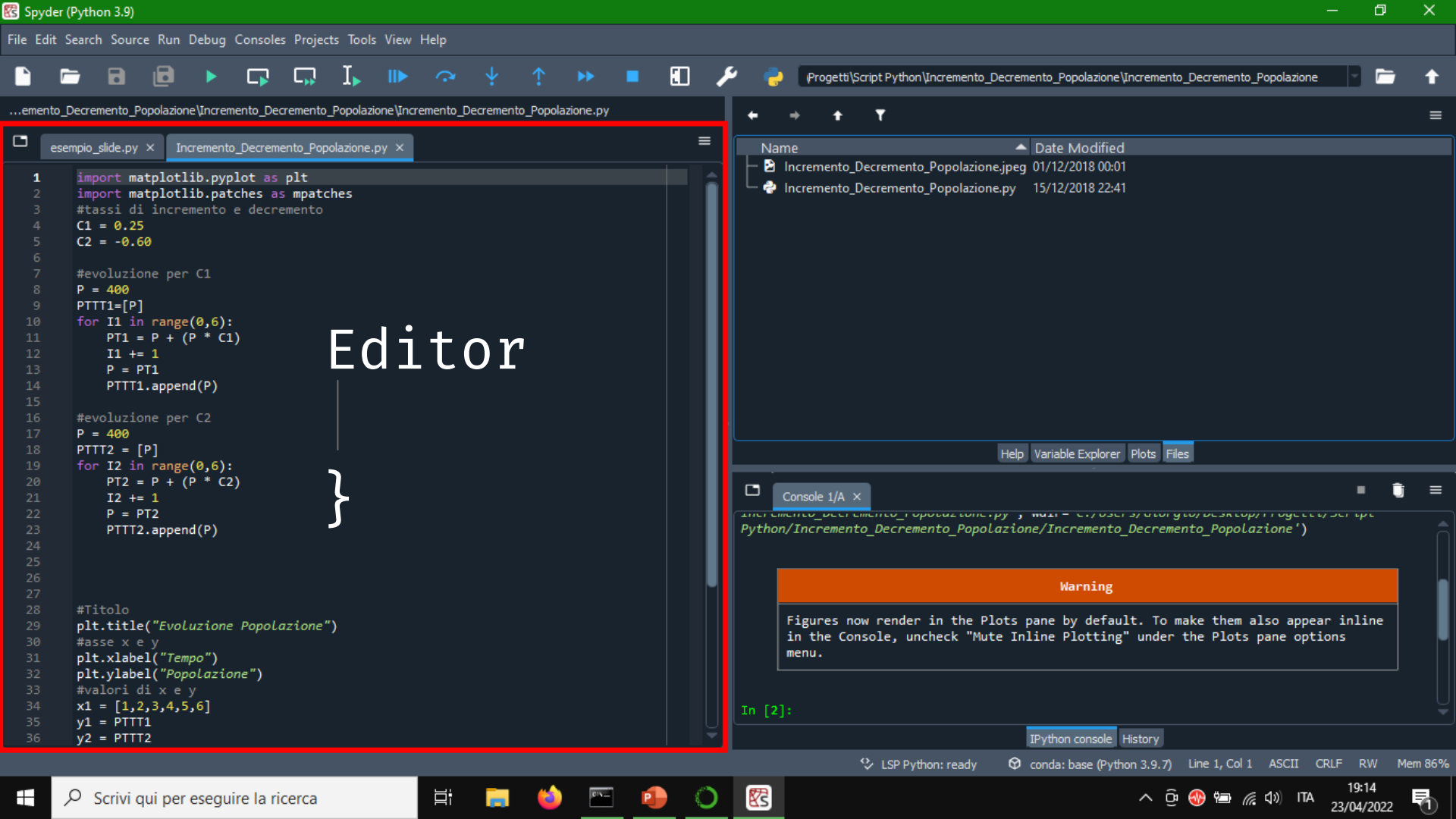
Spyder

5.1.5

Scientific PYTHON Development Environment. Powerful Python IDE with advanced editing, interactive testing, debugging and introspection features

Launch





Editor

}

Warning

Figures now render in the Plots pane by default. To make them also appear inline in the Console, uncheck "Mute Inline Plotting" under the Plots pane options menu.

In [2]:

Spyder (Python 3.9)

File Edit Search Source Run Debug Consoles Projects Tools View Help

Progetti\Script Python\Incremento_Decremento_Popolazione\Incremento_Decremento_Popolazione

...remento_Decremento_Popolazione\Incremento_Decremento_Popolazione\Incremento_Decremento_Popolazione.py

esempio_slide.py x Incremento_Decremento_Popolazione.py x

```
1 import matplotlib.pyplot as plt
2 import matplotlib.patches as mpatches
3 #tassi di incremento e decremento
4 C1 = 0.25
5 C2 = -0.60
6
7 #evoluzione per C1
8 P = 400
9 PTTT1=[P]
10 for I1 in range(0,6):
11     PT1 = P + (P * C1)
12     I1 += 1
13     P = PT1
14     PTTT1.append(P)
15
16 #evoluzione per C2
17 P = 400
18 PTTT2 = [P]
19 for I2 in range(0,6):
20     PT2 = P + (P * C2)
21     I2 += 1
22     P = PT2
23     PTTT2.append(P)
24
25
26
27
28 #Titolo
29 plt.title("Evoluzione Popolazione")
30 #asse x e y
31 plt.xlabel("Tempo")
32 plt.ylabel("Popolazione")
33 #valori di x e y
34 x1 = [1,2,3,4,5,6]
35 y1 = PTTT1
36 y2 = PTTT2
```

Console

Name	Date Modified
Incremento_Decremento_Popolazione.jpeg	01/12/2018 00:01
Incremento_Decremento_Popolazione.py	15/12/2018 22:41

Help Variable Explorer Plots Files

Console 1/A x

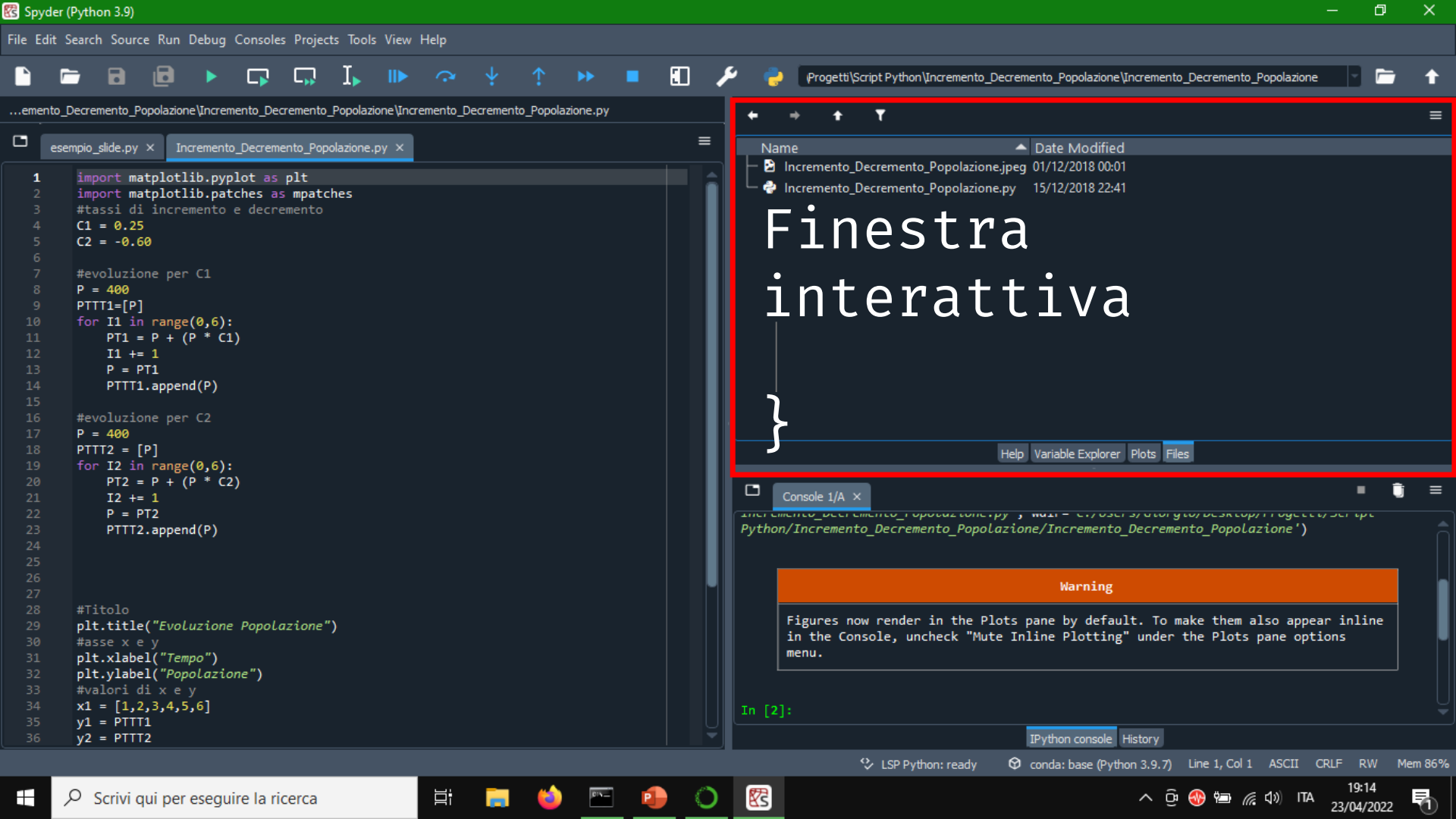
Incremento_Decremento_Popolazione.py , Path = C:\Users\stefano\Desktop\Progetti\Script Python\Incremento_Decremento_Popolazione\Incremento_Decremento_Popolazione')

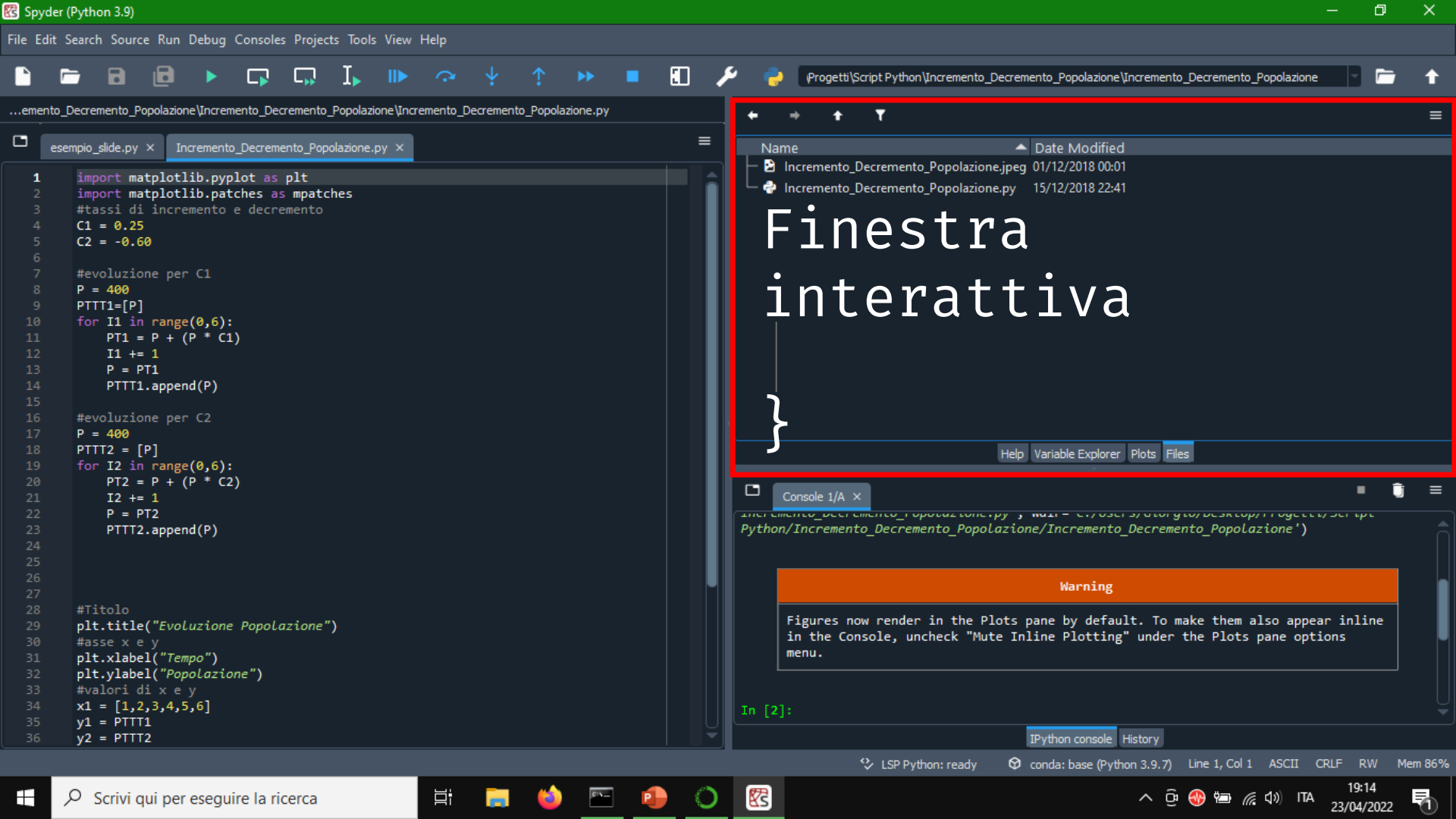
Warning

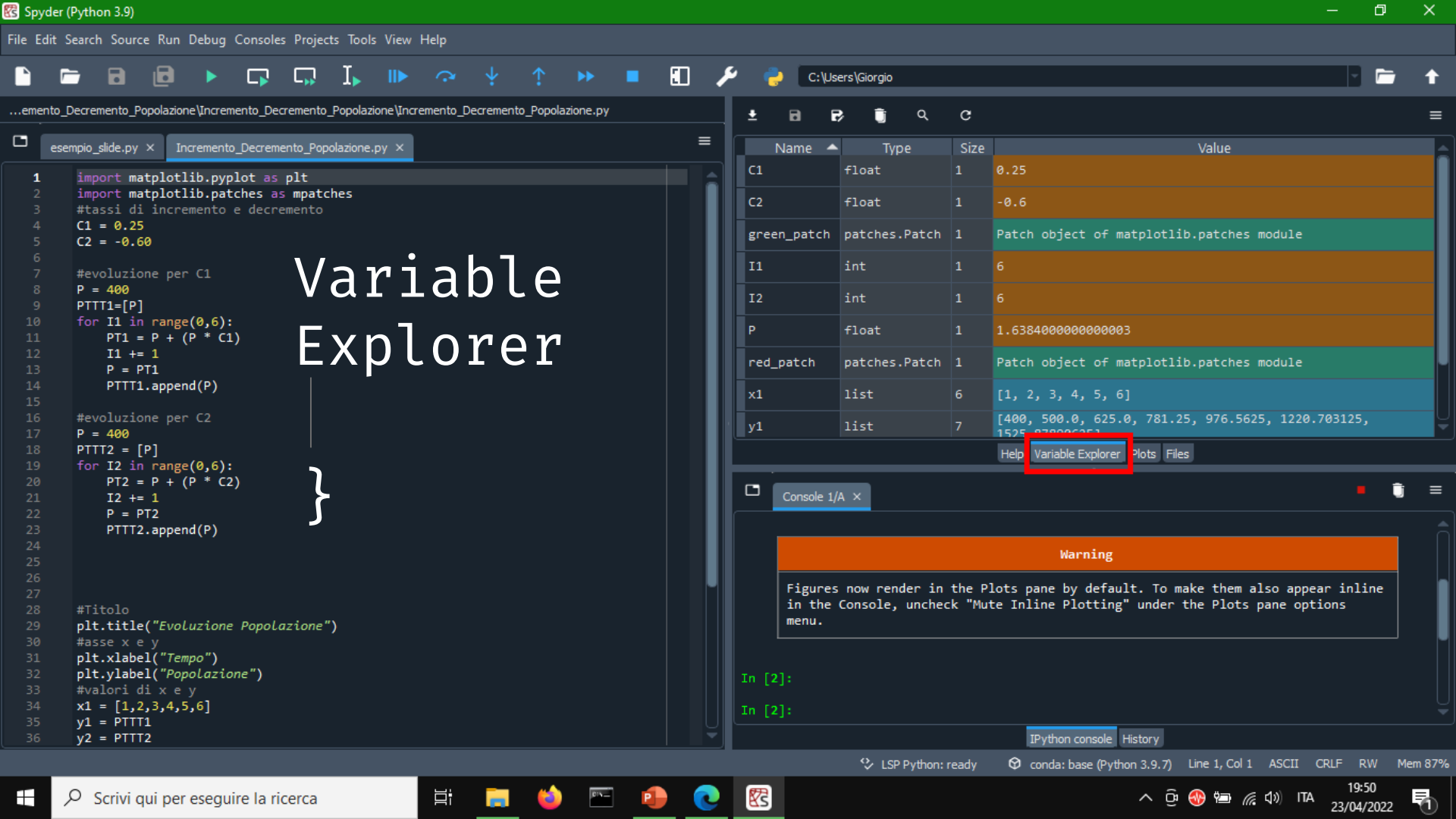
Figures now render in the Plots pane by default. To make them also appear inline in the Console, uncheck "Mute Inline Plotting" under the Plots pane options menu.

In [2]:

IPython console History







Variable Explorer

}

Name	Type	Size	Value
C1	float	1	0.25
C2	float	1	-0.6
green_patch	patches.Patch	1	Patch object of matplotlib.patches module
I1	int	1	6
I2	int	1	6
P	float	1	1.6384000000000003
red_patch	patches.Patch	1	Patch object of matplotlib.patches module
x1	list	6	[1, 2, 3, 4, 5, 6]
y1	list	7	[400, 500.0, 625.0, 781.25, 976.5625, 1220.703125, 1525.87890625]

Help Variable Explorer Plots Files

Warning

Figures now render in the Plots pane by default. To make them also appear inline in the Console, uncheck "Mute Inline Plotting" under the Plots pane options menu.

In [2]:

In [2]:

IPython console History

LSP Python: ready conda: base (Python 3.9.7) Line 1, Col 1 ASCII CRLF RW Mem 87%

19:50

23/04/2022

Spyder (Python 3.9)

File Edit Search Source Run Debug Consoles Projects Tools View Help

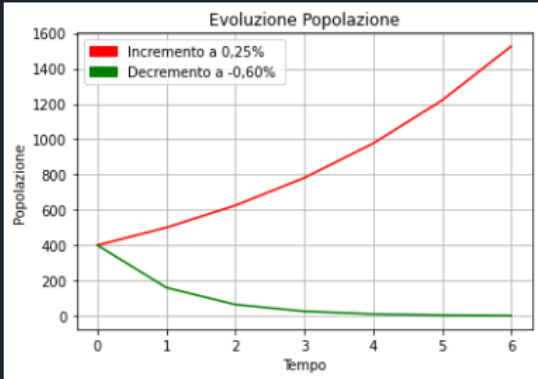
Progetti\Script Python\Incremento_Decremento_Popolazione\Incremento_Decremento_Popolazione

...emento_Decremento_Popolazione\Incremento_Decremento_Popolazione\Incremento_Decremento_Popolazione.py

esempio_slide.py x Incremento_Decremento_Popolazione.py x

```
1 import matplotlib.pyplot as plt
2 import matplotlib.patches as mpatches
3 #tassi di incremento e decremento
4 C1 = 0.25
5 C2 = -0.60
6
7 #evoluzione per C1
8 P = 400
9 PTTT1=[P]
10 for I1 in range(0,6):
11     PT1 = P + (P * C1)
12     I1 += 1
13     P = PT1
14     PTTT1.append(P)
15
16 #evoluzione per C2
17 P = 400
18 PTTT2 = [P]
19 for I2 in range(0,6):
20     PT2 = P + (P * C2)
21     I2 += 1
22     P = PT2
23     PTTT2.append(P)
24
25
26
27
28 #Titolo
29 plt.title("Evoluzione Popolazione")
30 #asse x e y
31 plt.xlabel("Tempo")
32 plt.ylabel("Popolazione")
33 #valori di x e y
34 x1 = [1,2,3,4,5,6]
35 y1 = PTTT1
36 y2 = PTTT2
```

Plot



Popolazione

Tempo

Incremento a 0,25%
Decremento a -0,60%

Help Variable Explorer Plots Files

Console 1/A x

Incremento_Decremento_Popolazione.py ; Path = C:\Users\stefano\Desktop\Progetti\Script Python\Incremento_Decremento_Popolazione\Incremento_Decremento_Popolazione\

Warning

Figures now render in the Plots pane by default. To make them also appear inline in the Console, uncheck "Mute Inline Plotting" under the Plots pane options menu.

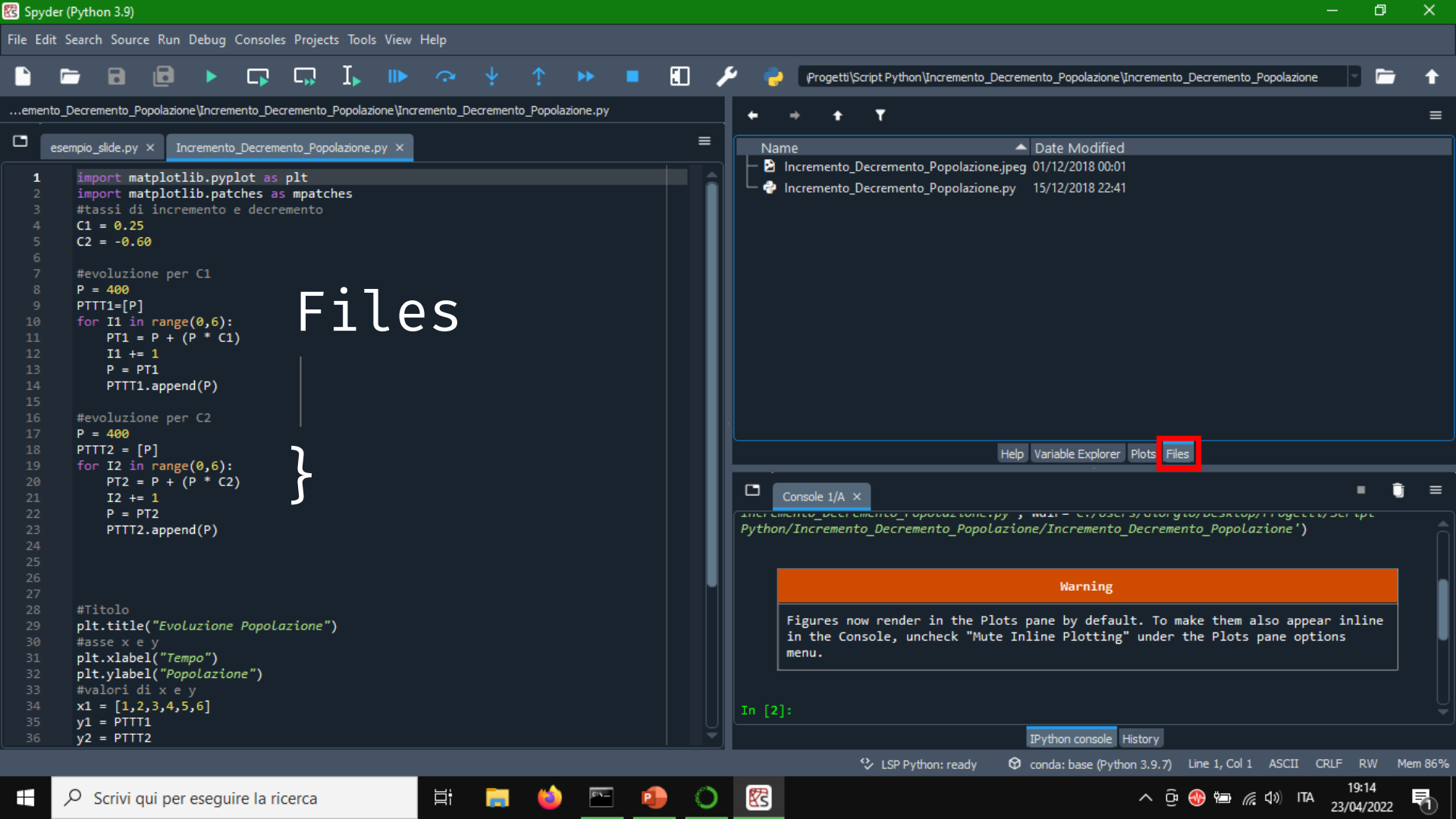
In [2]:

IPython console History

LSP Python: ready conda: base (Python 3.9.7) Line 1, Col 1 ASCII CRLF RW Mem 81%

Scrivi qui per eseguire la ricerca

19:14
23/04/2022

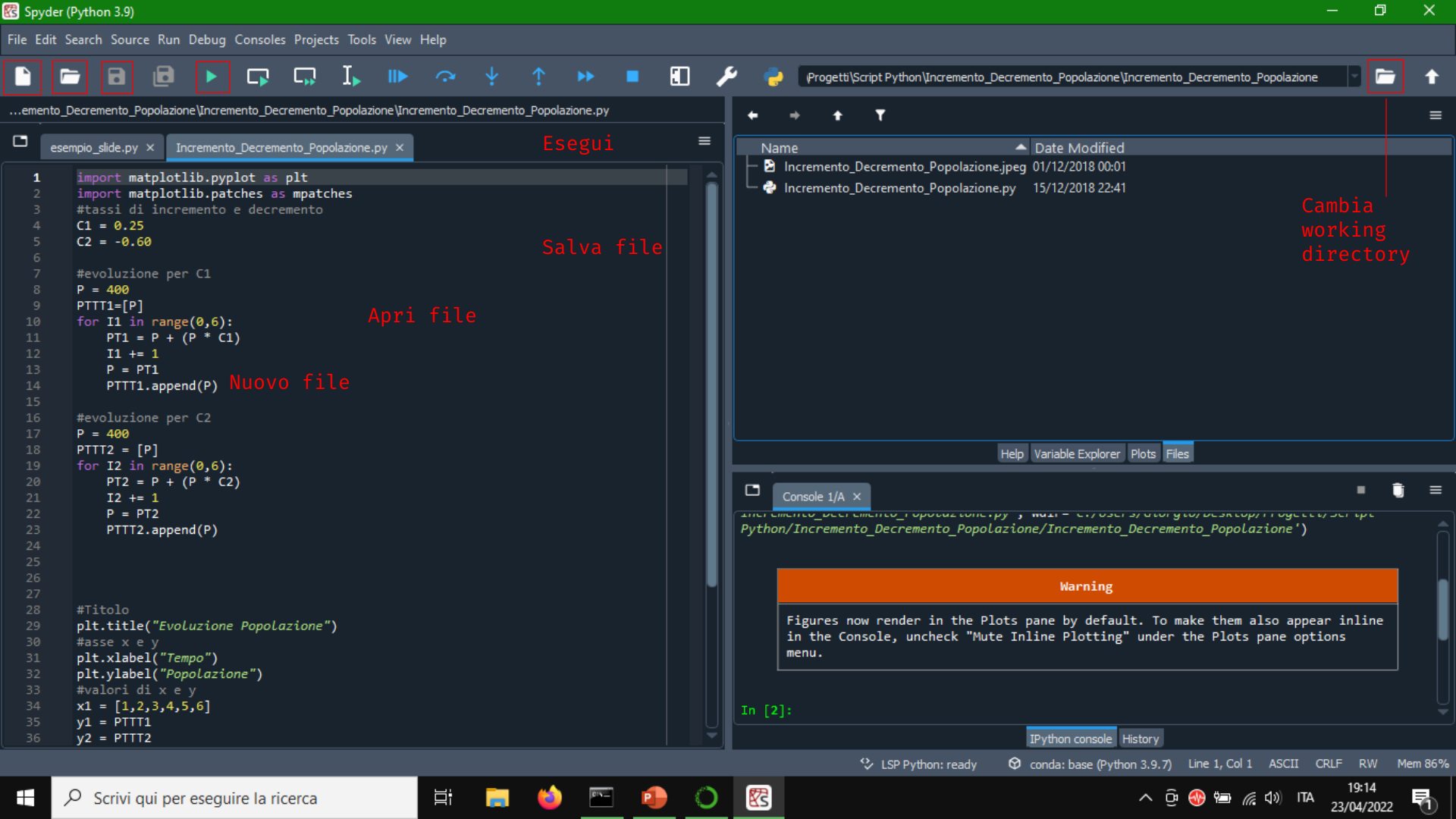


Files

Warning

Figures now render in the Plots pane by default. To make them also appear inline in the Console, uncheck "Mute Inline Plotting" under the Plots pane options menu.

In [2]:



Interfaccia Matlab in Spyder {

E' possibile passare dall'interfaccia base di Spyder a quella di Matlab

Per farlo: View -> Window layout -> Matlab layout

}

Name	Dal
> __pycache__	01/01/2019
> immagini	26/01/2019
calcolo_minimo_rappresentabile.py	15/01/2019
calcolo_precisione_macchina.py	14/01/2019
esempio_interpolazione.py	01/01/2019
esercizio_grafici.py	01/01/2019
esercizio1_intercorso.py	26/01/2019
interpolazione_esercizio_1.py	01/01/2019
interpolazione_esercizio_3.py	01/01/2019
interpolazione_esercizio_5.py	25/01/2019
interpolazione_esercizio_dati_discreti.py	01/01/2019

Runge_Equidistanti.py

Runge_Equidistanti.py x

```

1 #import delle librerie
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 plt.close('all') #chiusura automatica della finestra
6
7 #definizione della funzione
8 def f(x):
9     return 1./(1+np.square(x))
10
11 numero_nodi = [5,9,16] #lista contenente il numero di nodi
12 colori = ['red','green','blue'] #lista contenente i colori
13
14 a = -5 #limite inferiore
15 b = 5 #limite superiore
16 i = 0 #variabile utilizzata per accedere agli elementi della lista colori
17
18 x = np.linspace(a,b,1000) #punti delle ascisse

```

Console 1/A ✕

```
Type "copyright", "credits" or "license()" for more information.

IPython 7.29.0 -- An enhanced Interactive Python.

In [1]:

In [1]:

In [1]: runfile('C:/Users/Giorgio/Desktop/Università/CS/file/Programmi Python/
Runge_Equidistanti.py', wdir='C:/Users/Giorgio/Desktop/Università/CS/file/
Programmi Python')

In [2]:
```

Name	Type	Size	Value
a	int	1	-5
b	int	1	5
colori	list	3	['red', 'gree...
i	int	1	3
index	int	1	14
label_p	str	5	p(16)
label_punti	str	26	16 punti di interpolazione
nod_i	int	1	16
numero_nodi	list	3	[5, 9, 16]
p	Array of float64	(16,)	[-3.83747449e...
x	Array of float64	(1000,)	[-5. ...

Variable Explorer Plots

 history.py

```
runfile('C:/Users/Giorgio/Desktop/Università/CS/file/Programmi Python/Runge_Equidistanti.py', wdir='C:/Users/Giorgio/Desktop/Università/CS/file/Programmi Python')

## ---(Wed Jun 1 22:17:29 2022)---
runfile('C:/Users/Giorgio/Desktop/Università/CS/file/Programmi Python/Runge_Equidistanti.py', wdir='C:/Users/Giorgio/Desktop/Università/CS/file/Programmi Python')
```

History Help

Variabili {

Le variabili sono riferimenti a oggetti Python.

}

Variabili - 1 {

Sono create tramite **assegnamento**, ad esempio:

```
>>> a = 1
>>> b = 'Pippo'
>>> c = 88.93
>>> variabile = True
>>> |
```

E' possibile usare l'assegnamento multiplo, che permette di assegnare valori a più variabili sulla stessa linea di codice

```
>>> a,b,variabile = 3,'ciao',99.99
>>> a
3
>>> b
'ciao'
>>> variabile
99.99
>>> |
```

}

Variabili - 2 {

Le variabili hanno un nome che può essere una combinazione di lettere, numeri e carattere `_`, senza vincoli sulla lunghezza. Il nome di una variabile non può cominciare per un numero. I nomi sono **case sensitive**: la variabile `a` non è la variabile `A`

```
>>> a = 6
>>> A = 5
>>> a
6
>>> A
5
>>> |
```

E' buona norma dare un nome descrittivo alle variabili, in modo da tenere traccia del loro uso nel codice.

}

Variabili – 3 {

Python ha 33
parole riservate
che non possono
essere usate
come nome per le
variabili

and	as	assert	break	class	continue	def	del
elif	else	except	exec	False	finally	for	from
global	if	import	in	is	lambda	None	nonlocal
not	or	pass	raise	return	True	try	while
yield							

}

Tipi di dati {

}

Tipi di dati {

Python supporta 3 tipi di dati:

Numerici: usati per rappresentare i numeri. Si dividono in:
interi, che rappresenta i numeri interi (int)
reali, che rappresenta i numeri reali (float)
complessi, che rappresenta i numeri complessi (complex)

Booleani: valori che possono essere solo True o False (bool)

Stringhe: usati per rappresentare testo (str)

}

}

Alcuni esempi:

[illegible]

Numeri reali {

I numeri reali (float) sono rappresentati mediante **notazione decimale** oppure **notazione scientifica**.
Ad esempio:

3.78 può essere scritto come $3.78e0$ (3.78×10^0)

0.378 può essere scritto come $3.78e-1$ (3.78×10^{-1})

Il simbolo **e** separa l'esponente dalla mantissa.

```
>>> a = 9.0
>>> b = 7.89
>>> b11 = 7.89e0
>>> x = 541.98771
>>> v = 541e-3
>>> |
```

Numeri complessi {

I numeri immaginari in Python sono rappresentati mediante l'aggiunta di una `j` ad un numero reale, ad esempio:

```
>>> z = 5.2j
>>> |
```

I numeri complessi sono la somma di un numero reale e uno immaginario, ad esempio:

```
>>> z = 5.2j
>>> a = 3.5 + 5.2j
>>> |
```

}

Numeri complessi – Attenzione! {

Esempi:

```
b = 5.2
```

```
z = bj # restituisce NameError
```

```
z = b*1j #corretto
```

```
>>> b = 5.2
```

```
>>> z = b*j
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#34>", line 1, in <module>
```

```
    z = b*j
```

```
NameError: name 'j' is not defined
```

```
>>> z = b*1j #corretto
```

```
>>> |
```

Booleani {

I booleani possono assumere solo due valori: True o False e sono usati principalmente nelle espressioni logiche

```
>>> a = True
>>> b = False
>>>
```

}

Stringhe {

Una stringa è racchiusa tra doppie virgolette " " oppure doppi apici ' '.

Se la stringa è composta di più righe, puoi racchiuderla tra tre doppie virgolette """ oppure tre apici '''

```
>>> stringaa = 'pippo'
>>> a = "Paperino"
>>> stringa_su_piu_righe = '''sto scrivendo
su più righe!!!!'''
>>> stringa_su_piu_righe
'sto scrivendo\nsu più righe!!!!'
>>> _altra_stringa = """sto scrivendo su più righe!
Ho iniziato una nuova riga!
Altr riga."""
>>> _altra_stringa
'sto scrivendo su più righe!\nHo iniziato una nuova riga!\nAltr riga.'
>>> |
```

Tipi di dati{

[Funzione type()]

La funzione type() permette di ottenere il tipo di una variabile.

```
>>> a = 'i'
>>> b = 99.099
>>> z = 3j
>>> type(a)
<class 'str'>
>>> type(b)
<class 'float'>
>>> type(z)
<class 'complex'>
>>> o = 0
>>> type(o)
<class 'int'>
>>> boolean = True
>>> type(boolean)
<class 'bool'>
>>> |
```


Espressioni aritmetiche - 1 {

Un espressione aritmetica è formata da operandi e operatori disposti secondo le regole dell'algebra. Valgono anche le stesse regole di precedenza per valutare le espressioni aritmetiche.

OPERATORE	DESCRIZIONE	SINTASSI
+	Addizione	$a + b$
-	Sottrazione	$a - b$
*	Moltiplicazione	$a * b$
/	Divisione	a / b
//	Quoziente	$a // b$
%	Resto	$a \% b$
**	Potenza	$a ** b$

}

Espressioni aritmetiche - 2 {

Eseguire operazioni tra due int restituirà sempre un int.

Eseguire operazioni tra due float restituirà sempre un float.

Più in generale, quando si eseguono due operazioni su operandi dello stesso tipo, il risultato sarà sempre dello **stesso** tipo degli operandi.

Viceversa, se i due operandi sono diversi, il risultato sarà sempre del tipo più "generale" dei due.

}

Espressioni aritmetiche - 3 {

```
>>> a = 45
>>> b = 23.85
>>> z = 9.21 + 56j
>>> a + b
68.85
>>> type(a+b)
<class 'float'>
>>> c = 1
>>> type(a+c)
<class 'int'>
>>> a*b
1073.25
>>> type(a*b)
<class 'float'>
>>> |
```

```
>>> a + z
(54.21+56j)
>>> type(a+z)
<class 'complex'>
>>> b**a
9.704591338701827e+61
>>> type(b**a)
<class 'float'>
>>> -a
-45
>>> |
```

}

Concatenazione e ripetizione di stringhe {

Python permette di concatenare e ripetere una stringa usando gli operatori + e *

```
>>> nome = 'Giorgio'
>>> cognome = 'Esposito'
>>> nome_cognome = nome+cognome
>>> nome_cognome
'GiorgioEsposito'
>>> nome_cognome = nome + ' ' + cognome
>>> nome_cognome
'Giorgio Esposito'
>>> nome*5
'GiorgioGiorgioGiorgioGiorgioGiorgio'
>>> nome_cognome*3
'Giorgio EspositoGiorgio EspositoGiorgio Esposito'
>>> |
```

Attenzione: non si possono concatenare stringhe e numeri! Per poterlo fare bisogna convertire il numero in stringa tramite il metodo `str()` e poi concatenare il risultato alla stringa

Espressioni logiche - 1 {

Le espressioni logiche coinvolgono due (o più) variabili, su cui si effettua un confronto e restituiscono sempre un valore booleano. I vari operatori di confronto sono:

OPERATORE	DESCRIZIONE	SINTASSI
<	Minore	a < b
>	Maggiore	a > b
<=	Minore o uguale	a <= b
>=	Maggiore o uguale	a >= b
==	Uguale	a == b
!=	Diverso	a != b

}

Espressioni logiche - 2 {

Python permette di combinare diverse espressioni logiche o variabili booleane mediante gli operatori **and**, **or** e **not**.

Gli operatori **and** e **or** sono binari, ovvero richiedono due espressioni logiche o variabili, mentre l'operatore **not** è unario, ovvero richiede una singola espressione logica o variabile

L'operatore **and** restituisce True se e solo se i due operandi sono True, False altrimenti.

L'operatore **or** restituisce True se e solo se uno dei due operandi è True, False altrimenti.

L'operatore **not** invece esegue la negazione logica: se l'operando ha valore True, restituisce False, se l'operando ha valore False, restituisce True.

}

Espressioni composte {

Le espressioni logiche possono essere combinate per formare le espressioni composte. Ad esempio:

```
>>> a = 45
>>> b = 56.91
>>> c = 1.23
>>> d = 0.87
>>> (a <= b) and (c > d)
True
>>> (a == c) or (b > d)
True
>>> not(a == c)
True
>>> |
```

}

Short-circuit evaluation{

Python a volte può determinare il valore di un espressione senza dover calcolare il valore di ogni componente.
Ad esempio, supponiamo di avere

A and B

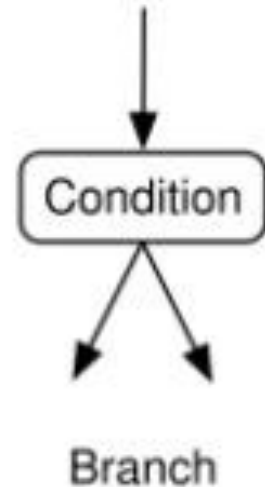
Se A è false allora l'intera espressione è falsa, non è necessario sapere quale sia il valore di B.
Questo meccanismo è detto **short-circuit evaluation** (**valutazione cortocircuito**)

}

Istruzione if - 1{

L'istruzione **if** è un'istruzione di selezione, ovvero serve a indicare al computer una scelta tra due diverse alternative. Anche per questo è detto **istruzione di selezione a due vie**.

}



Istruzione if – 2{

Questa è la sintassi di Python per il costrutto if – else

```
if (<condizione>):
```

```
    <istruzioni-1>
```

```
else:
```

```
    <istruzioni-2>
```

dove <condizione> è un'espressione logica semplice o composta.

```
}
```

Istruzione if – 3{

Alle volte può essere necessario inserire più di due condizioni quando si scrive un blocco if – else.

In questo caso si può utilizzare l'istruzione **elif**.

La sintassi in questo caso è:

```
If<condizione1>:  
    <istruzione1>  
elif<condizione2>:  
    <istruzione2>  
...  
elif<condizioneN>:  
    <istruzioneN>  
else:  
    <istruzione>
```

Istruzione if – esempio {

```
if(a < b):  
    a = b +1  
elif(a > c):  
    c = a  
elif(b > c):  
    b = 0  
else:  
    a = 0
```

```
}
```

Attenzione! {

01



Ogni sequenza di istruzioni all'interno del blocco if-elif-else deve essere rientrata di uno spazio rispetto ad if, elif ed else

02



Non inserire i due punti (:) dopo le istruzioni if, elif ed else causa un errore!

```
a = 0
b = 1
```

```
if(a < b):|
a = b + 1
```

```
File "<ipython-input-4-f19222194c66>", line 5
  a = b + 1
  ^
```

IndentationError: expected an indented block

```
a = 0
b = 1
```

```
if(a < b)
a = b + 1|
```

```
File "<ipython-input-2-716e552c6b1a>", line 4
  if(a < b)
    ^
```

SyntaxError: invalid syntax

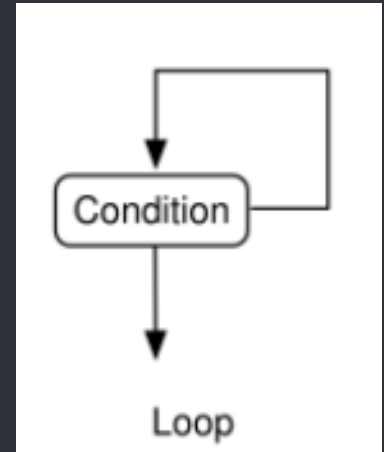
Cicli{

Alle volte può essere necessario dover eseguire una determinata istruzione un certo numero di volte. Per fare questo, Python (come la maggior parte dei linguaggi di programmazione) mette a disposizione i cicli.

Abbiamo due tipi di cicli:

- * Ciclo while
- * Ciclo for

}



Ciclo while – 1{

Il ciclo while è tipicamente utilizzato quando non è possibile determinare il numero di iterazioni di un ciclo: in questo caso il ciclo termina solo se la condizione diventa falsa: in questo caso si parla di **iterazione condizionale**.

La sintassi è la seguente:

```
while <condizione>:  
    <istruzioni>
```

```
}
```

Ciclo while - 2{

Il ciclo while controlla ad ogni iterazione la condizione specificata:

- * Se è falsa, il ciclo termina
- * Se è vera, il ciclo continua e le istruzioni specificate vengono eseguite di nuovo

Ovviamente all'interno del ciclo deve essere presente un'istruzione che rende falsa la condizione del ciclo. Se ciò non accade il ciclo non termina e si parla di **ciclo infinito**.

Il ciclo while è anche detto **ciclo a controllo iniziale**, perché la sua condizione viene controllata all'inizio del ciclo. Quindi le istruzioni possono essere eseguite zero o più volte.

}

Ciclo while - esempio{

```
a = 5  
while a > 0:  
    a -= 1  
    print(a)
```

```
4  
3  
2  
1  
0
```

```
}
```

Ciclo for – 1{

Il ciclo for è tipicamente utilizzato quando dobbiamo eseguire un'operazione certo numero di volte oppure per iterare sugli elementi di una lista.

La sintassi è la seguente:

```
for <variabile> in range(start, stop, step):  
    <istruzione1>  
    .....  
    <istruzioneN>
```

```
for <variabile> in sequenza:  
    <operazioni su variabile>
```

```
}
```

Ciclo for – esempi {

```
for x in range(0,4):  
    print(x)
```

```
0  
1  
2  
3
```

```
x = [1,2,3,4,5]
```

```
for elemento in x:  
    print(elemento)
```

```
1  
2  
3  
4  
5
```

```
}
```

Attenzione! ⚠️ {

01



Ogni sequenza di istruzioni all'interno dei cicli while e for deve essere indentata di uno spazio.

02



Non inserire i due punti (:) dopo l'intestazione del ciclo darà un errore!

```
for x in range(0,4):  
print(a)
```

```
File "<ipython-input-11-0cd21f6e2bb5>", line 2  
    print(a)  
      ^  
IndentationError: expected an indented block
```

```
a = 5
```

```
while a > 0  
    a -= 1  
    print(a)
```

```
File "<ipython-input-10-fd2f28037a5c>", line 3  
    while a > 0  
      ^  
SyntaxError: invalid syntax
```

Funzione range{

Nei cicli for controllati da contatori viene usata la funzione **range**.
La funzione range ha la seguente sintassi:

```
range(<start>,<stop + 1>, <step>)
```

Infatti la funzione range crea una lista di numeri che va dal limite inferiore fino al limite superiore, con un intervallo tra i numeri specificato dallo <step> (opzionale, default 1).

Ad esempio:

```
>>> for x in range(0,5):  
    print(x)
```

```
0  
1  
2  
3  
4
```

```
>>> |
```

```
>>> for x in range(0,9,2):  
    print(x)
```

```
0  
2  
4  
6  
8
```

```
>>> |
```

```
>>> for x in range(5,0,-1):  
    print(x)
```

```
5  
4  
3  
2  
1
```

```
>>> |
```

Input – 1 {

Se si vuole creare un programma che chiede input all'utente, bisogna utilizzare la funzione input.

La funzione interrompe l'esecuzione del programma e resta in attesa fino a quando l'utente non inserisce qualcosa. Quando l'utente preme il tasto Invio, la funzione accetta l'input e lo mette a disposizione del programma.

All'interno della funzione è possibile passare una stringa che indica cosa il programma vuole in input. Ad esempio:

```
nome = input('Inserisci il tuo nome: ')
```

}

Input – 2 {

La funzione `input` restituisce sempre una stringa, il che può creare problemi se si stanno inserendo valori numerici. Per questo, è possibile combinare la funzione `input` con le funzioni di conversione per trasformare la stringa in numero.

Le più utilizzate sono la funzione `int` e `float`. Ad esempio:

```
numero_intero = int(input('Inserisci un numero: '))  
numero_reale = float(input('Inserisci un numero: '))
```

}

Output – 1 {

La funzione `print()` permette di visualizzare il contenuto di una variabile, permettendo di specificare anche un messaggio:

```
>>> b = 7
>>> print(b)
7
>>> print("Valore di b:",b)
Valore di b: 7
>>> |
```

Tra il messaggio e la variabile va messa una virgola (,). Non farlo causa un errore

}

Output – 2 {

Python, a differenza di Matlab, non ha formati di stampa specificati. Ma, quando usiamo la funzione di output, possiamo utilizzare la formattazione per specificare il numero di cifre che possono essere rappresentate.

}

Output – 3 {

Come il linguaggio C, Python permette di formattare l'output usando %:

```
>>> a = 234.12309728991009234
>>> print("%2.3f" % a)
234.123
>>> nome = 'Giorgio'
>>> print("Ciao %s" % nome)
Ciao Giorgio
>>> eta = 21
>>> print("%s ha %d anni" % (nome, eta))
Giorgio ha 21 anni
>>>
```

La sintassi tipica è 'stringa' % (valori), dove le occorrenze di % in 'stringa' sono rimpiazzate da zero o più elementi di valori. Questa operazione è chiamata interpolazione di stringa.

}

Output – 4 {

Ecco alcuni operatori di formattazione:

%s -> stringa

%d -> interi

%f -> numeri reali/virgola mobile

%.<numero di cifre>f -> numeri in virgola mobile con <numero di
cifre dopo la virgola

%e -> notazione scientifica

}

Output – 5 {

Benché l'interpolazione di stringa sia molto facile da utilizzare, Python permette altri due modi per formattare l'output:

- * utilizzando il metodo `format()` delle stringhe
- * utilizzando le f-string (Formatted literal string)

}

Metodo format() {

Il metodo format() formatta il valore specifico e li inserisce al posto di un placeholder.

Il placeholder è definito all'interno di {} e può essere identificato usando dei nomi, indici oppure entrambi (possono essere lasciati vuoti).

```
>>> print("{} ha {} anni".format("Giorgio",21))
Giorgio ha 21 anni
>>> print("{nome} ha {eta}".format(nome = "Giorgio",eta = 21))
Giorgio ha 21
>>> print("{0} ha {1} anni".format("Giorgio",21))
Giorgio ha 21 anni
>>> print("{0} ha {1} anni".format(21,"Giorgio"))
21 ha Giorgio anni
>>> |
```

}

Metodo format() {

All'interno del placeholder è possibile aggiungere un 'tipo di formattazione' che permette di formattare il testo:

TIPO	DESCRIZIONE
:<	Allinea a sinistra
:>	Allinea a destra
:^	Allinea al centro
:=	Posiziona il segno del numero il più a sinistra possibile
:+	Usa un segno + per indicare se il risultato è positivo o negativo
:-	Usa un - per indicare se il risultato è negativo
:,	Usa la , come separatore delle migliaia
:_	Usa _ come separatore delle migliaia

}

Formatted Literal String {

Le Formatted Literal String hanno il prefisso `f` o `F` e consentono di inserire delle espressioni Python nella stringa, racchiudendole dentro le parentesi graffe `{}`

L'espressione può essere seguita da una sintassi che specifica la formattazione da applicare: questo permette un maggiore controllo su come il valore verrà formattato.

Ad esempio:

```
>>> print(f'Mi chiamo {nome}')  
Mi chiamo Giorgio  
>>> |
```

}

Comando import{

Il comando import è usato per importare e usare le librerie all'interno del programma.

Ad esempio:

```
>>> import math
>>>
>>> math.pi
3.141592653589793
>>> math.sin(90)
0.8939966636005579
>>> math.pow(2,2)
4.0
>>> |
```

}

Comando `import`

Col comando `import` viene importata tutta la libreria, ma se necessitiamo di determinate funzioni, possiamo usare il comando

```
from <nome-libreria> import <nome-funzione>
```

Ad esempio:

```
>>> from sys import float_info
>>> float_info
sys.float_info(max=1.7976931348623157e+308, max_exp=1024, max_10_exp=308,
20446049250313e-16, radix=2, rounds=1)
>>> |
```

Così facendo si potranno utilizzare solo le funzioni importate, le altre presenti nelle librerie NON sono utilizzabili.

}

Comando `import`

Opzionalmente, quando importiamo una libreria, possiamo dare un nome in modo tale da migliorare la leggibilità del codice. Tale operazione è effettuata tramite la seguente sintassi:

```
import <libreria> as <nome_da_me_definito>
```

Ad esempio:

```
import numpy as np
import matplotlib.pyplot as plt
```

}

Import di funzioni {

Inoltre, col comando `import` è possibile importare funzioni da programmi da noi realizzati

Ad esempio, supponiamo di avere nel programma `nodicheby.py` la funzione `nodidiChebychev(a,b,n)`

```
1 import numpy as np
2
3 def nodidiChebychev(a,b,n):
4
5     x = (a+b)/2 + (b-a)/2*(np.cos(((2*np.arange(0,n)+1)*np.pi/(2*n))))
6     return x
```

Possiamo ri-utilizzarla in un altro programma usando il comando `import`

```
4
5 #importiamo la funzione
6 from nodicheby import nodidiChebychev as nC
```

}

Import di funzioni – ATTENZIONE {

Quando si importa una funzione da un file.py, questo e il programma chiamante devono essere contenuti nella stessa directory

}

Comando help{

Come detto all'inizio, Python ha numerosissime librerie fatte di numerosissime funzioni. Quindi può essere facile non sapere cosa fa una funzione.

Il comando `help()` permette di ottenere una pagina di informazioni sul metodo o oggetto passato come parametro.

```
>>> help(print)
Help on built-in function print in module builtins:

print(...)
    print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

    Prints the values to a stream, or to sys.stdout by default.
    Optional keyword arguments:
    file: a file-like object (stream); defaults to the current sys.stdout.
    sep: string inserted between values, default a space.
    end: string appended after the last value, default a newline.
    flush: whether to forcibly flush the stream.

>>> |
```

Comando help{

Se non viene passato nessun parametro nel comando, viene inizializzato il comando di aiuto, dove puoi chiedere aiuto su tutto.

```
>>> help()

Welcome to Python 3.8's help utility!

If this is your first time using Python, you should definitely check out
the tutorial on the Internet at https://docs.python.org/3.8/tutorial/.

Enter the name of any module, keyword, or topic to get help on writing
Python programs and using Python modules.  To quit this help utility and
return to the interpreter, just type "quit".

To get a list of available modules, keywords, symbols, or topics, type
"modules", "keywords", "symbols", or "topics".  Each module also comes
with a one-line summary of what it does; to list the modules whose name
or summary contain a given string such as "spam", type "modules spam".

help> math
Help on built-in module math:

NAME
    math

DESCRIPTION
    This module provides access to the mathematical functions
    defined by the C standard.
```

Resources {



Link:

- * Python documentation: <https://docs.python.org/3.10/>



Riferimenti testuali:

- * C. Führer, **Scientific Computing with Python**

}

```
1 credits {
```

```
2  
3         Il template della presentazione è stato  
4         creato da Slidesgo https://slidesgo.com/
```

```
5  
6  
7  
8  
9         I contenuti sono stati sviluppati durante il corso di  
10        Calcolo Scientifico per Laurea Triennale in Informatica,  
11        docente prof. Dajana Conte, da Esposito Giorgio Angelo:  
12        https://github.com/giorgio-angelo-esposito
```

```
13  
14    }
```